ASSIGNMENT 5 – PART B – TASK B3

STATEMENT vs PREPAREDSTATEMENT vs BATCH

CO5 – Data Persistence Engineering | 8 Marks | Bloom's Level K4 – Analyse

1. Objective

Move bus-pass records into MySQL using JDBC and compare Statement, PreparedStatement and PreparedStatement batching. Verify 10-row correctness with SELECT and ResultSetMetaData, measure 1,000 / 10,000 / 100,000 rows, compare auto-commit with one transaction, and make an engineering recommendation.

2. Database schema

The supplied SQL creates bus_pass with pass_id VARCHAR(24) PRIMARY KEY, reg_no VARCHAR(20), name VARCHAR(80), route_no INT and amount DECIMAL(10,2). This follows the five columns in the supplied PassLoader starter code.

3. B3.1 – Correctness on 10 rows

The program clears the table and inserts 10 generated records using each of the three methods. It then selects all five columns, prints ResultSetMetaData column names, MySQL type names and Java class names, and verifies that exactly 10 rows exist.

Column	MySQL type	Java mapping
pass_id	VARCHAR(24)	String
reg_no	VARCHAR(20)	String
name	VARCHAR(80)	String
route_no	INT	Integer
amount	DECIMAL(10,2)	BigDecimal

4. B3.2 – Performance experiment

The Java program times all three approaches for 1,000, 10,000 and 100,000 records and prints CSV-formatted measurements. Actual times depend on the local machine, MySQL server, Connector/J version, indexes and storage. No timing values are fabricated here; enter the measurements from your own run.

Rows	Statement (ms)	Prepared (ms)	Batch (ms)
1,000	Fill from run	Fill from run	Fill from run
10,000	Fill from run	Fill from run	Fill from run
100,000	Fill from run	Fill from run	Fill from run

All approaches perform O(n) row work. Statement repeats SQL text construction and execution. PreparedStatement reuses a parameterized SQL structure. Batching groups b records per submission, reducing batch round trips to approximately ceil(n/b); pending client-side batch memory is O(b).

5. B3.3 – Auto-commit versus transaction

The failure test attempts 10,000 rows in batches of 1,000 and places an invalid amount at the first row of batch 6. With auto-commit enabled, the first five successful batches should remain committed (5,000 rows). With one transaction, the same failure should trigger rollback and leave zero rows.

Mode	Expected effect	Expected rows
Auto-commit on	Earlier successful batches remain	5,000
Single transaction	Rollback removes all batch inserts	0

These are expected outcomes for the test design. Confirm actual row counts from your MySQL execution and retain the console output.

6. B3.4 – Engineering trade-off

For a nightly import of 100,000 passes, PreparedStatement batching inside one transaction is preferred because it reduces round trips and gives all-or-nothing behaviour. For one pass issued at a counter, use a parameterized PreparedStatement with a single executeUpdate. Statement is a comparison baseline, not a good choice for concatenating untrusted values.

7. Complexity and alternatives

Statement and PreparedStatement perform O(n) insert operations. Batching remains O(n) total row work but reduces submissions to O(ceil(n/b)) and uses O(b) client memory for the pending batch. A multi-row INSERT can also reduce round trips but creates larger SQL/parameter payloads. MySQL LOAD DATA may be efficient for bulk imports, but needs careful file validation and secure server-side file handling.

8. Execution

Run sql/create_database_and_table.sql in MySQL Workbench. Add MySQL Connector/J to lib as mysql-connector-j.jar. Compile: javac -cp "lib\\mysql-connector-j.jar" -d output src\\PassLoader.java src\\B3Runner.java. Run: java -cp "output;lib\\mysql-connector-j.jar" B3Runner. Set DB_URL, DB_USER and DB_PASSWORD environment variables if required.

9. Conclusion

The task compares JDBC insertion correctness, throughput and failure recovery. PreparedStatement batching in one transaction is recommended for bulk imports; a single PreparedStatement is suitable for counter issuance. Complete the performance table and capture actual outputs from the local MySQL installation.

This report intentionally excludes student-identifying details. Actual timing results must be collected on the target MySQL setup; the report does not invent benchmark measurements.